

Experiment-10

AIM: Apply non-linear filters on images and investigate its application in noise-removal.

Theory:

Non-linear filters are image processing techniques used for noise removal by considering the local neighborhood of each pixel. Unlike linear filters, non-linear filters modify pixel values based on their relationship with neighboring pixels, allowing them to effectively suppress different types of noise.

Program

```
import cv2
import numpy as np

def process_image(image_path, window_name):
    """
    Process a single image with smoothing and sharpening
    """
    try:
        # Load the image
        image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

        if image is None:
            print(f"Error: Could not load image {image_path}")
            return

        print(f"Processing: {image_path}")
        print(f"Image shape: {image.shape}")

        # ----- 1. Original Image -----
        cv2.imshow(f"Original - {window_name}", image)

        # ----- 2. Smoothing -----
        # Step 1: Median Filter (remove salt & pepper noise)
        median_smoothed = cv2.medianBlur(image, 5)

        # Step 2: Bilateral Filter (smooth but keep edges)
        # Adjust parameters based on image size
        d = 9 # diameter of pixel neighborhood
        sigma_color = 75 # filter sigma in color space
        sigma_space = 75 # filter sigma in coordinate space

        smooth = cv2.bilateralFilter(median_smoothed, d, sigma_color, sigma_space)

        cv2.imshow(f"Smoothed - {window_name}", smooth)

        # ----- 3. Sharpening -----
        # Method 1: Unsharp Masking
        gaussian_blur = cv2.GaussianBlur(smooth, (0, 0), 3.0) # Using (0,0) to calculate from
sigma
        unsharp_mask = cv2.addWeighted(smooth, 1.5, gaussian_blur, -0.7, 0)
```

```

cv2.imshow(f"Sharpened (Unsharp) - {window_name}", unsharp_mask)

# Method 2: Laplacian Sharpening (alternative method)
laplacian = cv2.Laplacian(smooth, cv2.CV_64F)
laplacian_sharpened = cv2.convertScaleAbs(smooth - 0.7*laplacian)

cv2.imshow(f"Sharpened (Laplacian) - {window_name}", laplacian_sharpened)

# ----- 4. Enhanced Display -----
# Resize images for better display if they are too large
max_display_size = 800
height, width = image.shape

if max(height, width) > max_display_size:
    scale = max_display_size / max(height, width)
    new_width = int(width * scale)
    new_height = int(height * scale)

# Resize all images for consistent display
image_resized = cv2.resize(image, (new_width, new_height))
smooth_resized = cv2.resize(smooth, (new_width, new_height))
unsharp_resized = cv2.resize(unsharp_mask, (new_width, new_height))
laplacian_resized = cv2.resize(laplacian_sharpened, (new_width, new_height))

# Create a combined display
top_row = np.hstack((image_resized, smooth_resized))
bottom_row = np.hstack((unsharp_resized, laplacian_resized))
combined = np.vstack((top_row, bottom_row))

# Add labels to the combined image
combined_color = cv2.cvtColor(combined, cv2.COLOR_GRAY2BGR)
cv2.putText(combined_color, "Original", (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
0.7, (0, 255, 0), 2)
    cv2.putText(combined_color, "Smoothed", (new_width + 10, 30),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
    cv2.putText(combined_color, "Unsharp Mask", (10, new_height + 30),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
    cv2.putText(combined_color, "Laplacian", (new_width + 10, new_height + 30),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

cv2.imshow(f"Combined Results - {window_name}", combined_color)

print(f"Completed processing: {image_path}")
print("-" * 50)

except Exception as e:
    print(f"Error processing {image_path}: {str(e)}")

def main():

```

```

# List of image paths to process
image_paths = [
    "smoothing data.png",
    "smoothing data2.png",
    "smoothing data3.png",
    "smoothing data4.png",

]

print("Starting image processing...")
print("Press any key to move to the next image")
print("=" * 50)

# Process each image
for i, path in enumerate(image_paths, 1):
    process_image(path, f"Image {i}")

    # Wait for key press before showing next image
    key = cv2.waitKey(0)
    if key == 27: # ESC key to exit early
        print("Processing interrupted by user")
        break

    # Close all windows before next image
    cv2.destroyAllWindows()

print("All images processed!")
cv2.destroyAllWindows()

if __name__ == "__main__":
    main()

```

Output



